

UNIVERSIDAD AUTÓNOMA DE OCCIDENTE
Facultad de Ingeniería — Servicios Telemáticos
SEGUNDO PARCIAL — Documento de evidencias

Dato	Valor
Nombres	Santiago López López, Sebastian Bustamante Lopez, Byron Steven Zapata Zapata
Fecha de sustentación	6 de octubre de 2026
Repositorio GitHub	https://github.com/Santugoll/ParcialDos

Direccionamiento usado en mi entorno

El enunciado pide indicar el direccionamiento si es distinto al del diagrama. Complete/ajuste esta tabla si cambió algo.

Equipo	Hostname	Dirección(es)	Papel
Cliente	cli-2225727	192.168.56.20	Cliente Linux (también FileZilla en el PC)
Servidor 1	srv1-2225727	192.168.56.10 ("IP pública") / 192.168.50.3	Firewall UFW + DNAT
Servidor 2	srv2-2225727	192.168.50.2	vsftpd (FTPS) y OpenSSH (SFTP)

PRIMERA PARTE — FTPS protegido por Firewall UFW (2.0 puntos)

Servicio 1: Firewall y reenvío de puertos con UFW (Servidor 1) — 1.0 pt

Punto 1 (0.3) — UFW como único punto de entrada, DNAT y Servidor 2 inalcanzable

Política deny (entrante y routed), reenvío IP habilitado, DNAT hacia 192.168.50.2. Demostrar que el Servidor 2 no es alcanzable directo.

CAPTURA 1.1 — Políticas por defecto y reenvío IP

Estado de UFW y del reenvío IP en el Servidor 1.

Dónde / comando:

```
srv1: sudo ufw status verbose
srv1: sysctl net.ipv4.ip_forward
```

Debe verse en la captura:

- Status: active
- Default: deny (incoming), allow (outgoing), deny (routed)
- net.ipv4.ip_forward = 1

```
vagrant@srv1-2225727:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)

192.168.50.2 21/tcp ALLOW FWD Anywhere
192.168.50.2 50000:50010/tcp ALLOW FWD Anywhere
192.168.50.2 22/tcp ALLOW FWD Anywhere

vagrant@srv1-2225727:~$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

CAPTURA 1.2 — Reglas NAT (DNAT) en before.rules

Bloque `*nat` al inicio del archivo, antes de `*filter`.

Dónde / comando:

```
srv1: sudo head -20 /etc/ufw/before.rules
```

Debe verse en la captura:

- Líneas `*nat`, `:PREROUTING ACCEPT [0:0]` y `COMMIT`
- DNAT del puerto 21 y del rango 50000:50010 hacia 192.168.50.2

```
vagrant@srv1-2225727:~$ sudo head -20 /etc/ufw/before.rules
*nat
:PREROUTING ACCEPT [0:0]
-A PREROUTING -d 192.168.56.10 -p tcp --dport 21 -j DNAT --to-destination 192.168.50.2:21
-A PREROUTING -d 192.168.56.10 -p tcp --dport 50000:50010 -j DNAT --to-destination 192.168.50.2
-A PREROUTING -d 192.168.56.10 -p tcp --dport 2222 -j DNAT --to-destination 192.168.50.2:22
COMMIT

#
# rules.before
#
# Rules that should be run before the ufw command line added rules. Custom
# rules should be added to one of these chains:
#   ufw-before-input
#   ufw-before-output
#   ufw-before-forward
#

# Don't delete these required lines, otherwise there will be errors
*filter
:ufw-before-input - [0:0]
```

CAPTURA 1.3 — Servidor 2 no alcanzable directamente

Desde el cliente, el acceso directo a 192.168.50.2 debe fallar.

Dónde / comando:

```
cli: ping -c2 -W2 192.168.50.2
cli: nc -zv -w3 192.168.50.2 21
```

```
vagrant@cli-2225727:~$ ping -c2 -W2 192.168.50.2
PING 192.168.50.2 (192.168.50.2) 56(84) bytes of data.
ping: Warning: invalid tv_usec -86316885459121945 us
ping: Warning: time of day goes back (-31833915821672 s), taking countermeasures
ping: Warning: invalid tv_usec -86316885459121524 us
ping: Warning: time of day goes back (-31833915821672 s), taking countermeasures
64 bytes from 192.168.50.2: icmp_seq=0 ttl=255 time=0.000 ms
wrong data byte #16 should be 0x10 but was 0xc0
#16      c0 a8 32 2 8 0 86 25 0 1 0 2 dc 71 c1 6a 0 0 0 0 8 28 d 0 0 0 0 0 10 11 12 13
#48      14 15 16 17 18 19 1a 1b
ping: Warning: invalid tv_usec -86316885459121498 us
ping: Warning: time of day goes back (-31833915821672 s), taking countermeasures
64 bytes from 192.168.50.2: icmp_seq=0 ttl=255 time=0.000 ms (DUP!)
wrong data byte #16 should be 0x10 but was 0xc0
#16      c0 a8 32 2 8 0 86 25 0 1 0 2 dc 71 c1 6a 0 0 0 0 8 28 d 0 0 0 0 0 10 11 12 13
#48      14 15 16 17 18 19 1a 1b

--- 192.168.50.2 ping statistics ---
2 packets transmitted, 1 received, +1 duplicates, 50% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.000/0.000/0.000/0.000 ms, pipe 3
vagrant@cli-2225727:~$ nc -zv -w3 192.168.50.2 21
nc: connect to 192.168.50.2 port 21 (tcp) timed out: Operation now in progress
```

Punto 2 (0.2) — Solo el tráfico estrictamente necesario

22/tcp en el Servidor 1; route 21/tcp y 50000:50010/tcp hacia 192.168.50.2. Sin puertos adicionales.

CAPTURA 2.1 — Reglas finales de UFW

Lista numerada de reglas: no debe haber nada más que lo pedido.

Dónde / comando:

```
srv1: sudo ufw status numbered
```

```
srv1: sudo ss -tlnp (puertos escuchando en el propio Servidor 1)
```

Debe verse en la captura:

- 22/tcp ALLOW IN
- 192.168.50.2 21/tcp ALLOW FWD
- 192.168.50.2 50000:50010/tcp ALLOW FWD
- Ningún otro puerto abierto

```
vagrant@srv1-2225727:~$ sudo ufw status numbered
Status: active

    To Action From
    --
[ 1] 22/tcp ALLOW IN Anywhere
[ 2] 192.168.50.2 21/tcp ALLOW FWD Anywhere
[ 3] 192.168.50.2 50000:50010/tcp ALLOW FWD Anywhere
[ 4] 192.168.50.2 22/tcp ALLOW FWD Anywhere
[ 5] 22/tcp (v6) ALLOW IN Anywhere (v6)

vagrant@srv1-2225727:~$ sudo ss -tlnp
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:* users:([systemd-resolve,pid=642,fd=14])
LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:([sshd,pid=767,fd=3])
LISTEN 0 128 [::]:22 [::]:* users:([sshd,pid=767,fd=4])
```

Punto 3 (0.3) — Control de acceso: sin regla falla, con regla funciona

Con UFW activo y sin la regla de reenvío del puerto 21, la conexión a <ip pública> debe fallar; al agregar la regla, debe funcionar.

CAPTURA 3.1 — SIN la regla route: la conexión falla

Antes de agregar ufw route, muestre que no hay regla de reenvío y que el cliente no se conecta.

Dónde / comando:

```
cli: nc -zv -w3 192.168.56.10 21
```

Debe verse en la captura:

- Estado de UFW sin reglas FWD
- Timeout / fallo en la conexión desde el cliente

```
vagrant@cli-2225727:~$ nc -zv -w3 192.168.56.10 21
nc: connect to 192.168.56.10 port 21 (tcp) timed out: Operation now in progress
```

CAPTURA 3.2 — Se agrega la regla route

Comando que agrega la regla de reenvío.

Dónde / comando:

```
srv1: sudo ufw route allow proto tcp from any to 192.168.50.2 port 21
```

```
srv1: sudo ufw route allow proto tcp from any to 192.168.50.2 port 50000:50010
```

Debe verse en la captura:

- Rule added para cada comando

```
vagrant@srv1-2225727:~$ sudo ufw status numbered
Status: active
```

	To	Action	From
	--	-----	----
[1]	22/tcp	ALLOW IN	Anywhere
[2]	192.168.50.2 21/tcp	ALLOW FWD	Anywhere
[3]	192.168.50.2 50000:50010/tcp	ALLOW FWD	Anywhere
[4]	192.168.50.2 22/tcp	ALLOW FWD	Anywhere
[5]	22/tcp (v6)	ALLOW IN	Anywhere (v6)

CAPTURA 3.3 — CON la regla route: la conexión funciona

Mismo comando del cliente, ahora exitoso.

Dónde / comando:

```
cli: nc -zv -w3 192.168.56.10 21
```

Debe verse en la captura:

- Connection to 192.168.56.10 21 port [tcp/ftp] succeeded!

```
vagrant@cli-2225727:~$ nc -zv -w3 192.168.56.10 21
Connection to 192.168.56.10 21 port [tcp/ftp] succeeded!
```

RESPUESTA — Diferencia entre `ufw allow` y `ufw route allow`

Explique sobre qué cadena de iptables actúa cada una y cuándo se usa cada una.

ufw allow permite tráfico destinado al propio servidor donde está instalado UFW, por lo que actúa sobre el tráfico de entrada hacia ese equipo. En cambio, ufw route allow permite tráfico que atraviesa el servidor y es reenviado hacia otra máquina, por lo que se utiliza cuando srv1 funciona como firewall/router. En este caso, ufw allow 22/tcp permite la administración SSH de srv1, mientras que ufw route allow permite que el tráfico dirigido a los servicios de srv2 atraviese srv1 después de aplicar el DNAT.

Punto 4 (0.2) — Salida de UFW y de las reglas NAT, y su propósito**CAPTURA 4.1 — ufw status verbose**

Salida completa.

Dónde / comando:

```
srv1: sudo ufw status verbose
```

Debe verse en la captura:

- Políticas por defecto
- Reglas 22/tcp y las dos reglas FWD

```
vagrant@srv1-2225727:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)

192.168.50.2 21/tcp ALLOW FWD Anywhere
192.168.50.2 50000:50010/tcp ALLOW FWD Anywhere
192.168.50.2 22/tcp ALLOW FWD Anywhere
```

CAPTURA 4.2 — Reglas NAT activas

Salida completa de la tabla NAT con contadores.

Dónde / comando:

```
srv1: sudo iptables -t nat -L -n -v
```

Debe verse en la captura:

- Cadena PREROUTING con las dos reglas DNAT
- Contadores de paquetes > 0 si ya probó (opcional pero recomendado)

```
vagrant@srv1-2225727:~$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 3 packets, 1196 bytes)
 pkts bytes target prot opt in out source destination
 2 120 DNAT tcp -- * * 0.0.0.0/0 192.168.56.10 tcp dpt:21 to:192.168.50.2:21
 0 0 DNAT tcp -- * * 0.0.0.0/0 192.168.56.10 tcp dpts:50000:50010 to:192.168
.50.2
 0 0 DNAT tcp -- * * 0.0.0.0/0 192.168.56.10 tcp dpt:2222 to:192.168.50.2:22

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination
```

RESPUESTA — Propósito de cada regla en el contexto FTPS

Complete la tabla (una fila por regla).

Regla	Propósito en el servicio FTPS
22/tcp ALLOW IN	Permite la administración SSH del Servidor 1 desde el cliente. Es el único puerto de entrada directo permitido en srv1.
route 21/tcp → 192.168.50.2	Permite que el tráfico del canal de control FTP atraviese srv1 y llegue al servicio vsftpd de srv2 en el puerto 21.
route 50000:50010/tcp → 192.168.50.2	Permite que el tráfico del canal de datos pasivo de FTPS atraviese srv1 hacia los puertos 50000–50010 de srv2.
DNAT 21 → 192.168.50.2:21	Traduce las conexiones dirigidas al puerto 21 de srv1 para que sean entregadas al puerto 21 de srv2, donde está ejecutándose vsftpd.
DNAT 50000:50010 → 192.168.50.2	Traduce el tráfico destinado al rango pasivo de srv1 hacia el mismo rango de puertos de srv2, permitiendo las transferencias de datos FTPS.

Servicio 2: FTPS con vsftpd (Servidor 2) — 1.0 pt**Punto 5 (0.2) — vsftpd en FTPS explícito con cifrado obligatorio**

force_local_logins_ssl=YES, force_local_data_ssl=YES, SSLv2/SSLv3 deshabilitados, certificado y clave de clase.

CAPTURA 5.1 — Configuración TLS de vsftpd

Parámetros activos (sin comentarios ni líneas vacías).

Dónde / comando:

```
srv2: grep -vE '^s*#|^s*$' /etc/vsftpd.conf
```

Debe verse en la captura:

- ssl_enable=YES, rsa_cert_file, rsa_private_key_file
- force_local_logins_ssl=YES y force_local_data_ssl=YES
- ssl_sslv2=NO y ssl_sslv3=NO

```
vagrant@srv2-2225727:~$ grep -vE '^s*#|^s*$' /etc/vsftpd.conf
listen=YES
listen_ipv6=NO
anonymous_enable=NO
local_enable=YES
write_enable=YES
chroot_local_user=YES
allow_writeable_chroot=YES
pam_service_name=vsftpd
ssl_enable=YES
rsa_cert_file=/etc/ssl/certs/servidor.crt
rsa_private_key_file=/etc/ssl/private/servidor.key
force_local_logins_ssl=YES
force_local_data_ssl=YES
ssl_sslv2=NO
ssl_sslv3=NO
ssl_tlsv1=NO
ssl_ciphers=HIGH
require_ssl_reuse=NO
pasv_enable=YES
pasv_min_port=50000
pasv_max_port=50010
pasv_address=192.168.56.10
```

CAPTURA 5.2 — Servicio activo y certificado de clase

El servicio corre y escucha en el 21; los archivos del certificado están en su sitio.

Dónde / comando:

```
srv2: systemctl status vsftpd --no-pager
srv2: sudo ss -tlnp | grep :21
srv2: ls -l /etc/ssl/certs/servidor.crt /etc/ssl/private/servidor.key
```

Debe verse en la captura:

- active (running)
- vsftpd escuchando en :21
- Permisos de la clave privada (600)

```
vagrant@srv2-2225727:~$ systemctl status vsftpd --no-pager
● vsftpd.service - vsftpd FTP server
   Loaded: loaded (/lib/systemd/system/vsftpd.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-10-05 23:55:41 UTC; 6min ago
     Process: 640 ExecStartPre=/bin/mkdir -p /var/run/vsftpd/empty (code=exited, status=0/SUCCESS)
    Main PID: 642 (vsftpd)
       Tasks: 1 (limit: 1099)
      Memory: 2.4M
         CPU: 24ms
    CGroup: /system.slice/vsftpd.service
            └─642 /usr/sbin/vsftpd /etc/vsftpd.conf

Warning: some journal files were not opened due to insufficient permissions.
vagrant@srv2-2225727:~$ sudo ss -tlnp | grep :21
LISTEN 0      32          0.0.0.0:*        0.0.0.0:21     users:(("vsftpd",pid=642,fd=3))
vagrant@srv2-2225727:~$ ls -l /etc/ssl/certs/servidor.crt /etc/ssl/private/servidor.key
ls: cannot access '/etc/ssl/private/servidor.key': Permission denied
-rwxr-xr-x 1 root root 1468 Oct  3 21:13 /etc/ssl/certs/servidor.crt
```

Punto 6 (0.2) — Modo pasivo coherente con UFW**CAPTURA 6.1 — Parámetros pasivos**

Las tres líneas pasivas en vsftpd.conf.

Dónde / comando:

```
srv2: grep pasv /etc/vsftpd.conf
```

Debe verse en la captura:

- pasv_min_port=50000, pasv_max_port=50010
- pasv_address=192.168.56.10

```
vagrant@srv2-2225727:~$ grep pasv /etc/vsftpd.conf
pasv_enable=YES
pasv_min_port=50000
pasv_max_port=50010
pasv_address=192.168.56.10
vagrant@srv2-2225727:~$ |
```

RESPUESTA — (a) ¿Por qué el rango pasivo debe coincidir con las reglas del firewall?

El rango pasivo configurado en vsftpd debe coincidir con los puertos permitidos y reenviados por el firewall porque las conexiones de datos de FTP pasivo utilizan esos puertos. Si vsftpd utiliza puertos que UFW o el DNAT no permiten, el canal de control puede establecerse correctamente, pero las transferencias de archivos fallarán. Por eso, en este caso, el rango `50000–50010` está configurado tanto en vsftpd como en las reglas de UFW y DNAT.

RESPUESTA — (b) ¿Por qué en FTPS el firewall no puede abrir dinámicamente los puertos de datos?

En FTP tradicional, el firewall puede utilizar el contenido del canal de control para identificar los puertos que se utilizarán para la transferencia de datos. En FTPS, ese canal está cifrado después de `AUTH TLS`, por lo que el firewall ya no puede inspeccionar los comandos FTP ni conocer dinámicamente qué puerto de datos se negociará. Por esta razón, se debe configurar previamente un rango de puertos pasivos y permitirlo explícitamente en el firewall.

RESPUESTA — (c) ¿Qué ocurre si `pasv\_address` no se configura detrás de NAT?

Si pasv_address no se configura correctamente detrás de NAT, el servidor puede anunciar al cliente una dirección IP interna que no es accesible desde donde se encuentra el cliente. En consecuencia, el cliente puede establecer el canal de control por el puerto 21, pero no podrá establecer correctamente el canal de datos para listar directorios o transferir archivos. Por eso se configura `pasv\_address=192.168.56.10`, que corresponde a la dirección de srv1 publicada hacia el cliente.

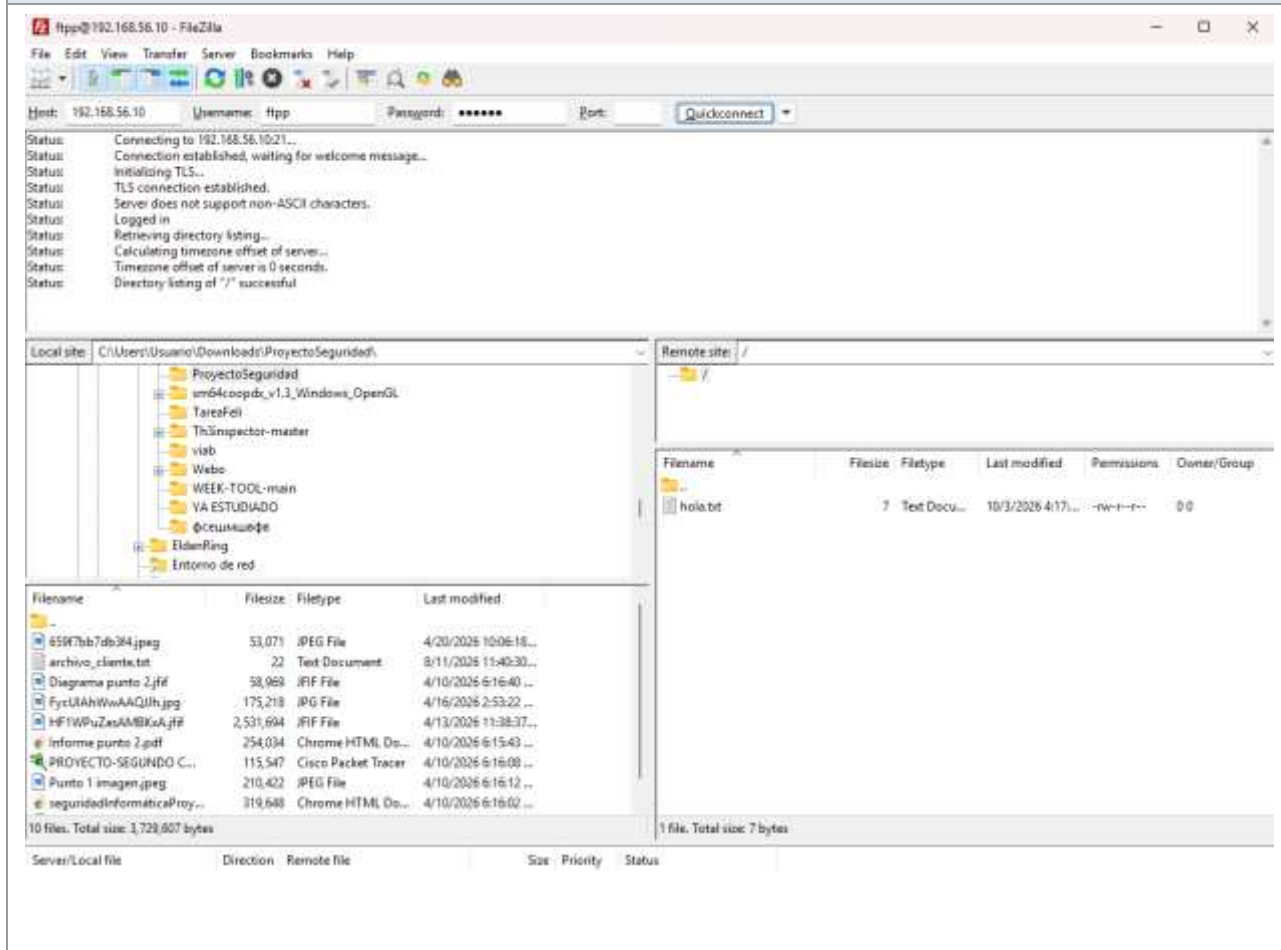
Punto 7 (0.2) — Conexión desde FileZilla (FTP explícito sobre TLS)

CAPTURA 7.1 — Configuración del sitio en FileZilla

Gestor de sitios con los datos de conexión.

Debe verse en la captura:

- Host 192.168.56.10, puerto 21
- Protocolo FTP, cifrado “Requerir FTP explícito sobre TLS”
- Usuario 2225727



RESPUESTA — Comparación de huellas

¿La huella de FileZilla coincide con la de OpenSSL? Describa el resultado y por qué es importante esta comparación.

La huella del certificado obtenida mediante OpenSSL es:

1A:E9:17:76:E0:B0:4D:63:48:5B:DD:20:82:28:3F:A6:75:7E:BE:9A:F7:DB:CA:39:78:8F:70:50:27:BA:E5:07

La huella coincide con la del certificado utilizado por FileZilla, por lo que se confirma que ambos están utilizando el mismo certificado del servidor. Esta comparación es importante porque permite verificar la identidad del servidor y detectar si el certificado fue reemplazado o si existe algún intento de conexión con un servidor diferente o no confiable.

CAPTURA 7.2 — Certificado presentado por el servidor

Ventana de “Certificado desconocido” / detalles del certificado.

Debe verse en la captura:

- Sujeto, emisor, vigencia (desde/hasta)
- Huella SHA-256 completa

```
vagrant@cli-2225727:~$ openssl x509 -in servidor.crt -noout -fingerprint -sha256
Could not open file or uri for loading certificate from servidor.crt
4067B4C1B47F0000:error:16000069:STORE routines:ossl_store_get0_loader_int:unregistered scheme:../crypto/store/store_regi
ster.c:237:scheme=file
4067B4C1B47F0000:error:80000002:system library:file_open:No such file or directory:../providers/implementations/storemgm
t/file_store.c:267:calling stat(servidor.crt)
Unable to load certificate
```

CAPTURA 7.3 — Huella calculada con OpenSSL

Comando en el servidor/cliente donde esté servidor.crt. Debe poder compararse a simple vista con la 7.2.

Dónde / comando:

```
openssl x509 -in servidor.crt -noout -fingerprint -sha256
```

Debe verse en la captura:

- Línea sha256 Fingerprint=XX:XX:...

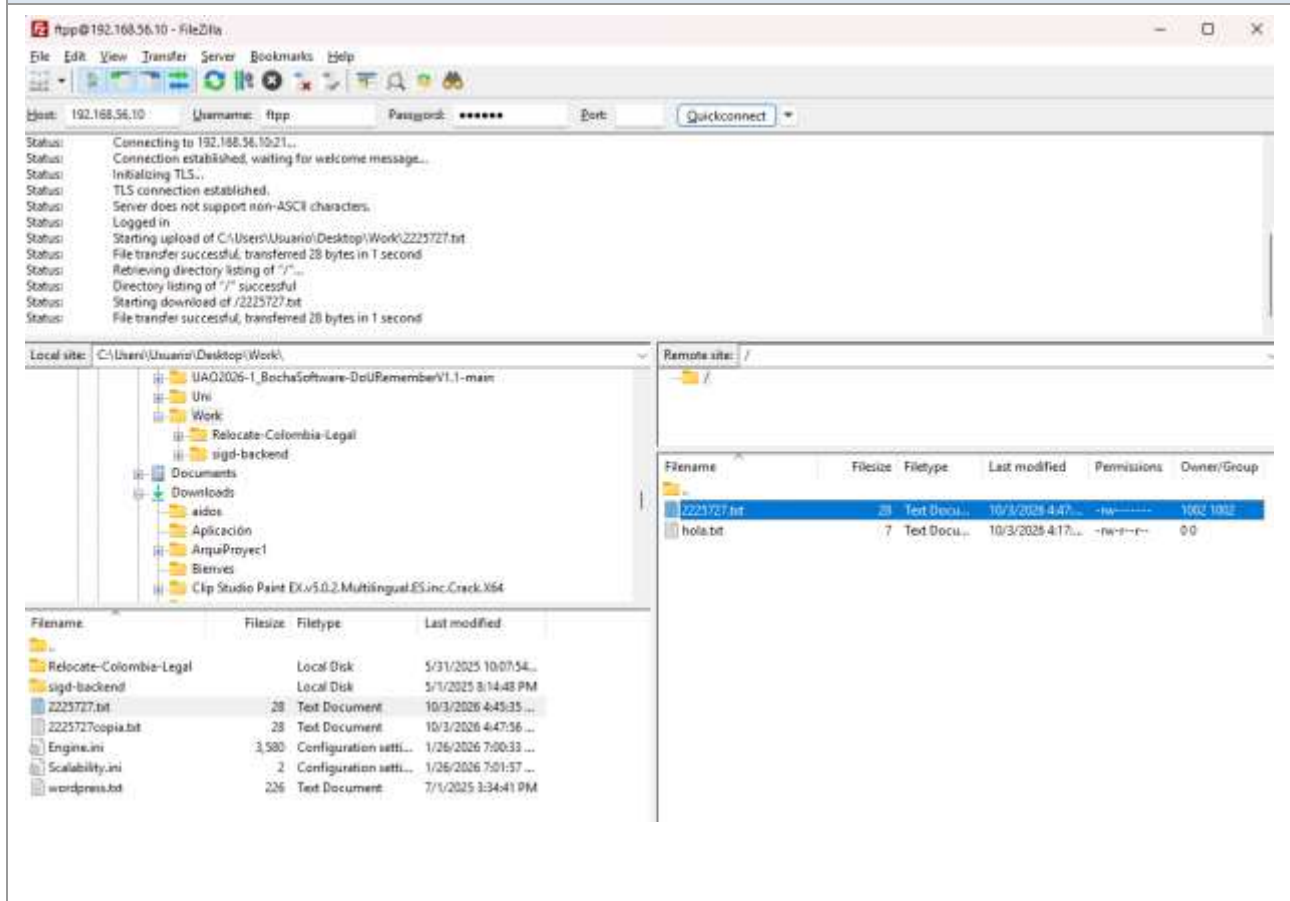
```
vagrant@srv2-2225727:~$ openssl x509 -in /vagrant/servidor.crt -noout -subject -issuer -dates -fingerprint -sha256
subject=CN = 192.168.56.10
issuer=CN = CA-2225727
notBefore=Oct  3 21:10:24 2026 GMT
notAfter=Oct  3 21:10:24 2027 GMT
sha256 Fingerprint=1A:E0:17:76:E0:80:4D:63:48:5B:DD:20:82:28:3F:A6:75:7E:BE:9A:F7:DB:CA:39:78:8F:70:50:27:8A:E5:07
```

CAPTURA 7.5 — Subida y descarga de 2225727.txt

Archivo subido al servidor y descargado de vuelta.

Debe verse en la captura:

- 2225727.txt en el panel remoto
- Cola de transferencias: subida y descarga exitosas
- Archivo descargado visible en el panel local
-



Punto 9 (0.2) — Capturas Wireshark: FTP plano vs FTPS

Conserve los archivos .pcap (ftp_plano.pcap y ftps.pcap): se usan en la Tercera Parte.

CAPTURA 9.1 — FTP plano: USER y PASS visibles

Filtro ftp.

Debe verse en la captura:

- Paquetes Request: USER 2225727 y Request: PASS ... con la clave legible

ftp_plano.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ftp.request.command == "USER"

No.	Time	Source	Destination	Protocol	Length	Info
6	3.718599	192.168.56.20	192.168.56.10	FTP	77	Request: USER ftp
46	60.628827	192.168.56.20	192.168.56.10	FTP	77	Request: USER ftp

ftp_plano.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ftp.request.command == "PASS"

No.	Time	Source	Destination	Protocol	Length	Info
10	6.624059	192.168.56.20	192.168.56.10	FTP	79	Request: PASS niidea
50	62.355742	192.168.56.20	192.168.56.10	FTP	79	Request: PASS niidea

CAPTURA 9.2 — FTP plano: contenido del archivo

Clic derecho en un paquete de datos → Follow → TCP Stream.

Debe verse en la captura:

- Contenido del archivo en texto plano

ftp_plano.pcap

Wireshark · Follow TCP Stream (tcp.stream eq 0) · ftp_plano.pcap

File Edit View

tcp.stream eq 0

No.	Time
1	0.0000
2	0.0007
3	0.0012
4	0.0044
5	0.0050
6	3.7185
7	3.7190
8	3.7196
9	3.7200
10	6.6240
11	6.6674
12	6.6678
13	6.6682
14	6.6689
15	6.6695
16	6.6700
17	6.6701
18	6.6702
19	6.6705
20	6.7210
21	27.577
22	27.579
23	27.579
27	27.580
28	27.582
34	27.583
35	27.583
36	45.844
37	45.845
38	45.845
39	45.846
40	45.846

```

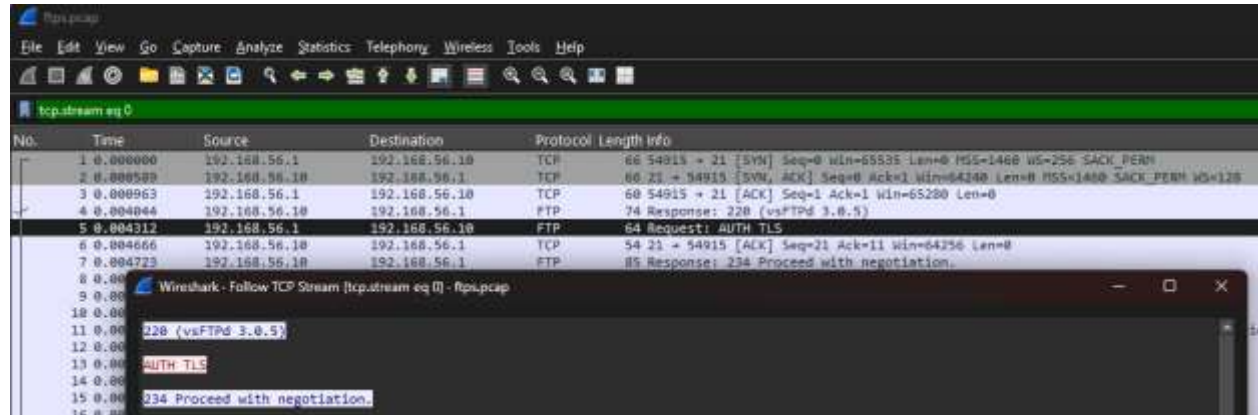
220 (vsFTPd 3.0.5)
USER ftpp
331 Please specify the password.
PASS niidea
230 Login successful.
SYST
215 UNIX Type: L8
FEAT
211-Features:
EPRT
EPSV
MDTM
PASV
REST STREAM
SIZE
TVFS
211 End
EPSV
229 Entering Extended Passive Mode (|||50003|)
LIST
150 Here comes the directory listing.
226 Directory send OK.
QUIT
221 Goodbye.
  
```

CAPTURA 9.3 — FTPS: comando AUTH TLS

Filtro ftp.

Debe verse en la captura:

- Request: AUTH TLS y Response: 234 ...

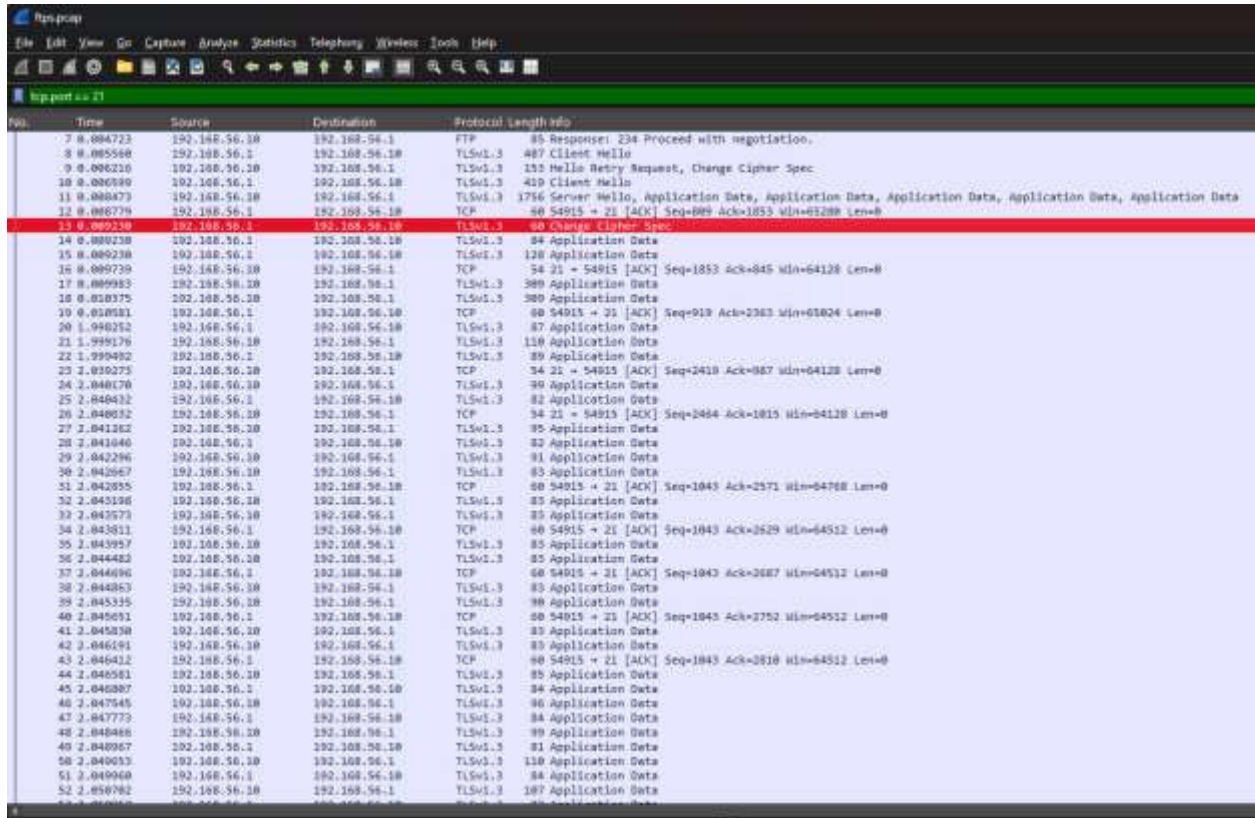


CAPTURA 9.4 — FTP: handshake TLS en el puerto 21

Filtro `tls && tcp.port == 21`.

Debe verse en la captura:

- Client Hello, Server Hello, Certificate



No.	Time	Source	Destination	Protocol	Length	Info
7	0.004723	192.168.56.10	192.168.56.1	FTP	85	Response: 234 Proceed with negotiation.
8	0.005568	192.168.56.1	192.168.56.10	TLSv1.3	487	Client Hello
9	0.006210	192.168.56.10	192.168.56.1	TLSv1.3	132	Hello Retry Request, Change Cipher Spec
10	0.006590	192.168.56.1	192.168.56.10	TLSv1.3	419	Client Hello
11	0.008473	192.168.56.10	192.168.56.1	TLSv1.3	1756	Server Hello, Application Data, Application Data, Application Data, Application Data, Application Data
12	0.008778	192.168.56.1	192.168.56.10	TCP	60	54915 → 21 [ACK] Seq=689 Ack=1853 Win=61200 Len=0
13	0.009230	192.168.56.10	192.168.56.1	TLSv1.3	120	Application Data
14	0.009238	192.168.56.1	192.168.56.10	TLSv1.3	84	Application Data
15	0.009238	192.168.56.1	192.168.56.10	TLSv1.3	120	Application Data
16	0.009730	192.168.56.10	192.168.56.1	TCP	54	21 → 54915 [ACK] Seq=1853 Ack=845 Win=64128 Len=0
17	0.009983	192.168.56.10	192.168.56.1	TLSv1.3	389	Application Data
18	0.010375	192.168.56.10	192.168.56.1	TLSv1.3	389	Application Data
19	0.010581	192.168.56.1	192.168.56.10	TCP	60	54915 → 21 [ACK] Seq=919 Ack=2383 Win=61024 Len=0
20	1.990252	192.168.56.1	192.168.56.10	TLSv1.3	87	Application Data
21	1.999176	192.168.56.10	192.168.56.1	TLSv1.3	110	Application Data
22	1.999492	192.168.56.1	192.168.56.10	TLSv1.3	89	Application Data
23	1.999275	192.168.56.10	192.168.56.1	TCP	54	21 → 54915 [ACK] Seq=2419 Ack=887 Win=64128 Len=0
24	2.040170	192.168.56.10	192.168.56.1	TLSv1.3	99	Application Data
25	2.040432	192.168.56.1	192.168.56.10	TLSv1.3	82	Application Data
26	2.040632	192.168.56.10	192.168.56.1	TCP	54	21 → 54915 [ACK] Seq=2464 Ack=1815 Win=64128 Len=0
27	2.041282	192.168.56.10	192.168.56.1	TLSv1.3	95	Application Data
28	2.041040	192.168.56.1	192.168.56.10	TLSv1.3	82	Application Data
29	2.042296	192.168.56.10	192.168.56.1	TLSv1.3	91	Application Data
30	2.042067	192.168.56.10	192.168.56.1	TLSv1.3	83	Application Data
31	2.042855	192.168.56.1	192.168.56.10	TCP	60	54915 → 21 [ACK] Seq=1043 Ack=2571 Win=64768 Len=0
32	2.045196	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
33	2.045573	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
34	2.045811	192.168.56.1	192.168.56.10	TCP	60	54915 → 21 [ACK] Seq=1043 Ack=2629 Win=64512 Len=0
35	2.045957	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
36	2.044482	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
37	2.044696	192.168.56.1	192.168.56.10	TCP	60	54915 → 21 [ACK] Seq=1043 Ack=2687 Win=64512 Len=0
38	2.044863	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
39	2.045335	192.168.56.10	192.168.56.1	TLSv1.3	90	Application Data
40	2.045651	192.168.56.1	192.168.56.10	TCP	60	54915 → 21 [ACK] Seq=1043 Ack=2752 Win=64512 Len=0
41	2.045830	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
42	2.046191	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
43	2.046412	192.168.56.1	192.168.56.10	TCP	60	54915 → 21 [ACK] Seq=1043 Ack=2810 Win=64512 Len=0
44	2.046561	192.168.56.10	192.168.56.1	TLSv1.3	85	Application Data
45	2.046807	192.168.56.1	192.168.56.10	TLSv1.3	94	Application Data
46	2.047545	192.168.56.10	192.168.56.1	TLSv1.3	90	Application Data
47	2.047773	192.168.56.1	192.168.56.10	TLSv1.3	84	Application Data
48	2.048488	192.168.56.10	192.168.56.1	TLSv1.3	99	Application Data
49	2.048967	192.168.56.1	192.168.56.10	TLSv1.3	81	Application Data
50	2.049033	192.168.56.10	192.168.56.1	TLSv1.3	110	Application Data
51	2.049960	192.168.56.1	192.168.56.10	TLSv1.3	84	Application Data
52	2.050792	192.168.56.10	192.168.56.1	TLSv1.3	107	Application Data

SEGUNDA PARTE — DNS sobre TLS (DoT) (1.5 puntos)

Punto 10 (0.3) — Configuración de systemd-resolved

CAPTURA 10.1 — resolved.conf

Contenido efectivo del archivo.

Dónde / comando:

```
cli: grep -vE '^s*#|^s*$' /etc/systemd/resolved.conf
```

Debe verse en la captura:

- DNS= con dos resolvers con nombre (1.1.1.1#cloudflare-dns.com 8.8.8.8#dns.google)
- FallbackDNS= coherente
- DNSOverTLS=yes

```
vagrant@cli-2225727:~$ grep -vE '^s*#|^s*$' /etc/systemd/resolved.conf
[Resolve]
DNS=1.1.1.1#cloudflare-dns.com 8.8.8.8#dns.google
FallbackDNS=1.0.0.1#cloudflare-dns.com 8.8.4.4#dns.google
DNSOverTLS=yes
Domains=~.
```

CAPTURA 10.2 — resolv.conf apunta al stub y reinicio del servicio**Dónde / comando:**`cli: ls -l /etc/resolv.conf && cat /etc/resolv.conf``cli: sudo systemctl restart systemd-resolved && systemctl status systemd-resolved --no-pager`**Debe verse en la captura:**

- nameserver 127.0.0.53
- Enlace a stub-resolv.conf
- Servicio active (running)

```
vagrant@cli-2225727:~$ ls -l /etc/resolv.conf && cat /etc/resolv.conf
lrwxrwxrwx 1 root root 37 Oct  3 22:29 /etc/resolv.conf -> /run/systemd/resolve/stub-resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .
vagrant@cli-2225727:~$ systemctl status systemd-resolved --no-pager
● systemd-resolved.service - Network Name Resolution
   Loaded: loaded (/lib/systemd/system/systemd-resolved.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-10-05 23:55:15 UTC; 20min ago
     Docs: man:systemd-resolved.service(8)
           man:org.freedesktop.resolve1(5)
           https://www.freedesktop.org/wiki/Software/systemd/writing-network-configuration-managers
           https://www.freedesktop.org/wiki/Software/systemd/writing-resolver-clients
   Main PID: 573 (systemd-resolve)
     Status: "Processing requests..."
    Tasks: 1 (limit: 1099)
   Memory: 7.3M
      CPU: 1.568s
   CGroup: /system.slice/systemd-resolved.service
           └─573 /lib/systemd/systemd-resolved

Warning: some journal files were not opened due to insufficient permissions.
vagrant@cli-2225727:~$
```

RESPUESTA — `DNSOverTLS=yes` vs `DNSOverTLS=opportunistic`

Explique la diferencia y qué ocurre en cada caso si el servidor no soporta TLS.

La diferencia está en qué sucede cuando el servidor DNS no soporta TLS. Con `DNSOverTLS=yes`, la conexión DNS debe realizarse mediante TLS, por lo que si el servidor no soporta DNS sobre TLS, la consulta no se realiza y la resolución DNS falla. En cambio, con `DNSOverTLS=opportunistic`, el sistema intenta utilizar TLS primero, pero si el servidor no lo soporta, puede continuar utilizando DNS sin cifrado. Por lo tanto, `yes` exige el uso de TLS, mientras que `opportunistic` permite continuar sin TLS cuando no está disponible.

Punto 11 (0.2) — resolvectl status**CAPTURA 11.1 — Salida completa de resolvectl status**

Resalte (recuadro rojo) la evidencia de DoT y de los servidores.

Dónde / comando:

```
cli: resolvectl status
```

Debe verse en la captura:

- `+DNSOverTLS` en `Protocols` o `DNSOverTLS setting: yes`
- `1.1.1.1#cloudflare-dns.com` y `8.8.8.8#dns.google`

```
vagrant@cli-2225727:~$ resolvectl status
Global
    Protocols: -LLMNR -mDNS +DNSOverTLS DNSSEC=no/unsupported
    resolv.conf mode: stub
    Current DNS Server: 1.1.1.1#cloudflare-dns.com
    DNS Servers: 1.1.1.1#cloudflare-dns.com 8.8.8.8#dns.google
    Fallback DNS Servers: 1.0.0.1#cloudflare-dns.com 8.8.4.4#dns.google
    DNS Domain: ~.

Link 2 (enp0s3)
    Current Scopes: DNS
    Protocols: +DefaultRoute +LLMNR -mDNS +DNSOverTLS DNSSEC=no/unsupported
    Current DNS Server: 8.8.8.8
    DNS Servers: 8.8.8.8 8.8.4.4 192.168.50.3 fd17:625c:f037:2::3

Link 3 (enp0s8)
    Current Scopes: none
    Protocols: -DefaultRoute +LLMNR -mDNS +DNSOverTLS DNSSEC=no/unsupported
```

RESPUESTA — Dónde se evidencia que DoT está activo

Señale la línea exacta y los servidores configurados.

DoT está evidenciado en la salida de `resolvectl status`, donde aparece `+DNSOverTLS` en la sección de protocolos. Además, se muestran los servidores configurados `1.1.1.1#cloudflare-dns.com` y `8.8.8.8#dns.google`, lo que confirma que systemd-resolved está configurado para utilizar DNS sobre TLS.

Punto 12 (0.2) — Resolución de tres dominios**CAPTURA 12.1 — Tres consultas por systemd-resolved**

Tres dominios distintos, sin @servidor.

Dónde / comando:

```
cli: resolvectl query uao.edu.co
cli: resolvectl query cloudflare.com
cli: dig github.com
```

Debe verse en la captura:

- Respuestas resueltas
- En resolvectl: -- Information acquired via protocol DNS ... encrypted: yes (si aparece)

```
vagrant@cli-2225727:~$ resolvectl query google.com
google.com: 172.217.30.206          -- link: enp0s3
           2800:3f0:4005:40c::200e  -- link: enp0s3

-- Information acquired via protocol DNS in 80.0ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: yes
-- Data from: network
vagrant@cli-2225727:~$ resolvectl query cloudflare.com
cloudflare.com: 2606:4700::6810:85e5 -- link: enp0s3
               2606:4700::6810:84e5 -- link: enp0s3
               104.16.132.229       -- link: enp0s3
               104.16.133.229       -- link: enp0s3

-- Information acquired via protocol DNS in 29.9ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: yes
-- Data from: network
vagrant@cli-2225727:~$ resolvectl query github.com
github.com: 140.82.114.3           -- link: enp0s3

-- Information acquired via protocol DNS in 30.6ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: yes
-- Data from: network
vagrant@cli-2225727:~$ |
```

RESPUESTA — ¿Por qué `dig @8.8.8.8 <dominio>` no usaría DoT?

`dig @8.8.8.8 <dominio>` realiza una consulta DNS directamente contra el servidor `8.8.8.8` utilizando el protocolo DNS convencional, normalmente por UDP/TCP en el puerto 53. Esta consulta no pasa por systemd-resolved y, por lo tanto, no utiliza la configuración `DNSOverTLS=yes`. Para utilizar DoT en este escenario se deben realizar las consultas mediante el resolutor configurado por systemd-resolved.

Punto 13 (0.5) — Capturas con DoT y sin DoT**CAPTURA 13.1 — Limpieza de caché y consulta con DoT activo**

Dónde / comando:

```
cli: resolvectl flush-caches && resolvectl query uao.edu.co
```

Debe verse en la captura:

- Comandos y respuesta exitosa

```
vagrant@cli-2225727:~$ resolvectl flush-caches && resolvectl query uao.edu.co
uao.edu.co: 104.26.9.14          -- link: enp0s3
              172.67.73.231      -- link: enp0s3
              104.26.8.14        -- link: enp0s3
              2606:4700:20::ac43:49e7 -- link: enp0s3
              2606:4700:20::681a:90e  -- link: enp0s3
              2606:4700:20::681a:80e  -- link: enp0s3

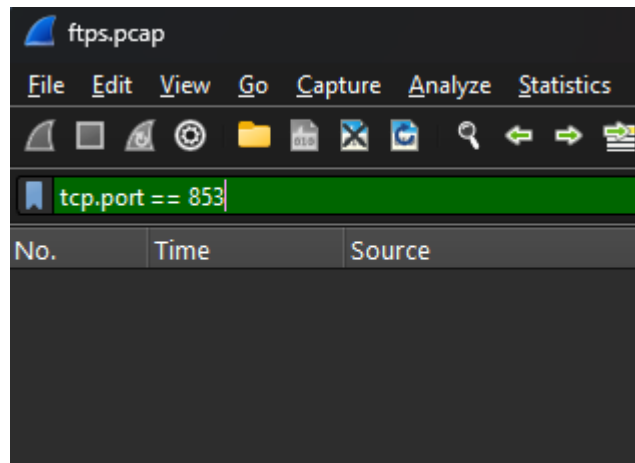
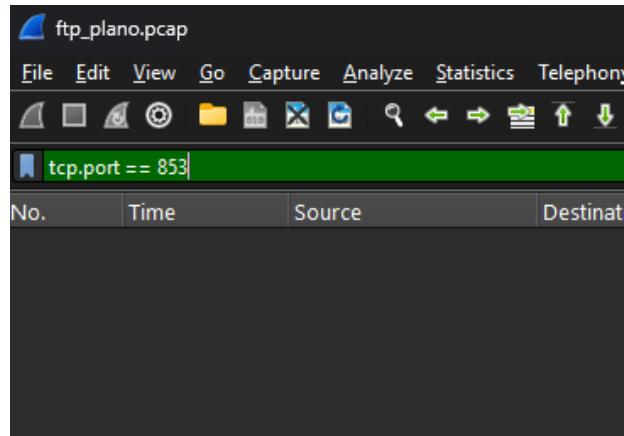
-- Information acquired via protocol DNS in 84.6ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: yes
-- Data from: network
vagrant@cli-2225727:~$ |
```

CAPTURA 13.2 — Wireshark con DoT

Filtro tcp.port == 853.

Debe verse en la captura:

- Handshake TLS (Client Hello con SNI del resolver)
- Paquetes Application Data cifrados; nada del dominio legible



CAPTURA 13.3 — DoT deshabilitado

Archivo con DNSOverTLS=no, reinicio y consulta.

Dónde / comando:

```
cli: grep DNSOverTLS /etc/systemd/resolved.conf
cli: sudo systemctl restart systemd-resolved
cli: resolvectl flush-caches && resolvectl query uao.edu.co
```

Debe verse en la captura:

- DNSOverTLS=no
- Consulta resuelta

```
vagrant@cli-2225727:~$ grep DNSOverTLS /etc/systemd/resolved.conf
DNSOverTLS=no
vagrant@cli-2225727:~$ sudo systemctl restart systemd-resolved
vagrant@cli-2225727:~$ resolvectl flush-caches && resolvectl query uao.edu.co
uao.edu.co: 2606:4700:20::ac43:49e7      -- link: enp0s3
            2606:4700:20::681a:80e      -- link: enp0s3
            2606:4700:20::681a:90e      -- link: enp0s3
            172.67.73.231                -- link: enp0s3
            104.26.8.14                  -- link: enp0s3
            104.26.9.14                  -- link: enp0s3

-- Information acquired via protocol DNS in 30.4ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: no
-- Data from: network
```

RESPUESTA — ¿Qué información queda expuesta en DNS sin cifrar?

Recuerde volver a poner DNSOverTLS=yes al terminar.

En DNS sin cifrar, un observador de la red puede identificar las consultas realizadas, incluyendo los nombres de dominio solicitados, además de la dirección IP del cliente y del servidor DNS y los momentos en que se realizan las consultas. También puede observar las respuestas DNS y, por tanto, obtener información sobre los dominios que está consultando el equipo. Esto permite conocer parte de la actividad de navegación aunque el contenido posterior de la conexión al sitio esté cifrado.

Punto 14 (0.3) — Análisis de DoT

RESPUESTA — Información visible para un observador con DoT activo

IP del resolver, SNI, tamaños y tiempos, puerto 853...

Con DoT activo, el contenido de las consultas DNS queda cifrado, por lo que un observador no puede leer directamente los nombres de dominio consultados ni las respuestas DNS. Sin embargo, todavía puede observar información de metadatos, como la dirección IP del resolver, el puerto TCP 853, los tamaños de los paquetes y los tiempos de las comunicaciones. También puede existir información como el SNI durante el establecimiento de TLS.

RESPUESTA — Firewall bloqueando el puerto 853: `yes` vs `opportunistic`

¿Qué pasa con la resolución en cada modo y qué implica para la seguridad?

Con `DNSOverTLS=yes`, systemd-resolved requiere utilizar DNS sobre TLS, por lo que si el puerto TCP 853 está bloqueado y no puede establecerse la conexión cifrada, la resolución DNS falla en lugar de recurrir automáticamente a DNS sin cifrar. En cambio, con `DNSOverTLS=opportunistic`, el sistema intenta utilizar DoT, pero si no puede establecer la conexión TLS puede continuar utilizando DNS convencional sin cifrado. Por seguridad, `yes` evita este retroceso y garantiza que las consultas no se envíen en claro.

RESPUESTA — DoT vs DoH en este escenario

Compare puerto, facilidad de bloqueo/identificación y privacidad.

DoT utiliza normalmente TCP 853, por lo que es relativamente sencillo de identificar y bloquear mediante un firewall que controle ese puerto. DoH transporta las consultas DNS mediante HTTPS, normalmente sobre el puerto 443, por lo que su tráfico se mezcla con el tráfico web HTTPS y resulta más difícil diferenciarlo y bloquearlo únicamente por puerto. Ambos protegen el contenido de las consultas DNS mediante cifrado, pero DoH ofrece una mayor facilidad para integrarse con redes donde el tráfico HTTPS ya está permitido.

CAPTURA 14.1 (opcional) — Demostración del bloqueo del 853

Si desea reforzar su respuesta: bloquee la salida al 853 en el cliente y muestre qué pasa con DNSOverTLS=yes.

Dónde / comando:

```
cli: sudo iptables -I OUTPUT -p tcp --dport 853 -j REJECT
cli: resolvectl flush-caches && resolvectl query uao.edu.co
cli: sudo iptables -D OUTPUT -p tcp --dport 853 -j REJECT (revertir)
```

Debe verse en la captura:

- Fallo de resolución con yes

```
vagrant@cli-2225727:~$ resolvectl flush-caches && resolvectl query uao.edu.co
uao.edu.co: resolve call failed: All attempts to contact name servers or networks failed
```

TERCERA PARTE — SFTP protegido por Firewall UFW (1.5 puntos)

Punto 15 (0.4) — OpenSSH con usuario exclusivo SFTP y chroot (Servidor 2)

CAPTURA 15.1 — Bloque Match User en sshd_config

Dónde / comando:

```
srv2: sudo tail -12 /etc/ssh/sshd_config  
srv2: sudo sshd -t && echo OK
```

Debe verse en la captura:

- Match User sftp_2225727, ChrootDirectory, ForceCommand internal-sftp
- OK (sintaxis válida)

```
vagrant@srv2-2225727:~$ sudo tail -12 /etc/ssh/sshd_config  
#Match User anoncvs  
#      X11Forwarding no  
#      AllowTcpForwarding no  
#      PermitTTY no  
#      ForceCommand cvs server  
  
Match User sftp_2225727  
    ChrootDirectory /srv/sftp/sftp_2225727  
    ForceCommand internal-sftp  
    PasswordAuthentication yes  
    AllowTcpForwarding no  
    X11Forwarding no  
vagrant@srv2-2225727:~$ sudo sshd -t && echo OK  
OK  
vagrant@srv2-2225727:~$ |
```

CAPTURA 15.2 — Estructura del chroot y permisos

El directorio chroot debe pertenecer a root.

Dónde / comando:

```
srv2: ls -ld /srv/sftp /srv/sftp/sftp_2225727 /srv/sftp/sftp_2225727/upload
```

Debe verse en la captura:

- root root en /srv/sftp y /srv/sftp/sftp_2225727
- upload propiedad de sftp_2225727

```
vagrant@srv2-2225727:~$ ls -ld /srv/sftp /srv/sftp/sftp_2225727 /srv/sftp/sftp_2225727/upload
drwxr-xr-x 3 root      root      4096 Oct  3 22:37 /srv/sftp
drwxr-xr-x 3 root      root      4096 Oct  3 22:37 /srv/sftp/sftp_2225727
drwxr-xr-x 2 sftp_2225727 sftp_2225727 4096 Oct  3 22:58 /srv/sftp/sftp_2225727/upload
```

CAPTURA 15.3 — SSH rechazado (sin shell)

Dónde / comando:

```
srv2: ssh sftp_2225727@localhost
```

Debe verse en la captura:

- Mensaje This service allows sftp connections only. y cierre de conexión

```
vagrant@srv2-2225727:~$ ssh sftp_2225727@localhost
sftp_2225727@localhost's password:
This service allows sftp connections only.
Connection to localhost closed.
vagrant@srv2-2225727:~$
```

CAPTURA 15.4 — SFTP sí funciona

Dónde / comando:

```
srv2: sftp sftp_2225727@localhost
sftp> ls
```

Debe verse en la captura:

- Sesión SFTP abierta y listado del directorio (upload)

```
vagrant@srv2-2225727:~$ sftp sftp_2225727@localhost
sftp_2225727@localhost's password:
Connected to localhost.
sftp> |
```

Punto 16 (0.3) — Reenvío 2222 → 192.168.50.2:22 en el Servidor 1

CAPTURA 16.1 — Regla DNAT del puerto 2222

Bloque *nat actualizado.

Dónde / comando:

```
srv1: sudo head -20 /etc/ufw/before.rules
srv1: sudo iptables -t nat -L PREROUTING -n -v
```

Debe verse en la captura:

- DNAT dpt:2222 to:192.168.50.2:22

```
vagrant@srv1-2225727:~$ sudo head -20 /etc/ufw/before.rules
#nat
:PREROUTING ACCEPT [0:0]
-A PREROUTING -d 192.168.56.10 -p tcp --dport 21 -j DNAT --to-destination 192.168.50.2:21
-A PREROUTING -d 192.168.56.10 -p tcp --dport 50000:50010 -j DNAT --to-destination 192.168.50.2
-A PREROUTING -d 192.168.56.10 -p tcp --dport 2222 -j DNAT --to-destination 192.168.50.2:22
COMMIT

#
# rules.before
#
# Rules that should be run before the ufw command line added rules. Custom
# rules should be added to one of these chains:
#   ufw-before-input
#   ufw-before-output
#   ufw-before-forward
#

# Don't delete these required lines, otherwise there will be errors
#filter
:ufw-before-input - [0:0]
vagrant@srv1-2225727:~$ sudo iptables -t nat -L PREROUTING -n -v
Chain PREROUTING (policy ACCEPT 15 packets, 3944 bytes)
  pkts bytes target     prot opt in     out     source               destination
    0     0 DNAT      tcp    --    *      *        0.0.0.0/0            192.168.56.10      tcp dpt:21 to:192.168.50.2:21
    0     0 DNAT      tcp    --    *      *        0.0.0.0/0            192.168.56.10      tcp dpts:50000:50010 to:192.168.50.2
    0     0 DNAT      tcp    --    *      *        0.0.0.0/0            192.168.56.10      tcp dpt:2222 to:192.168.50.2:22
vagrant@srv1-2225727:~$ |
```

CAPTURA 16.2 — SIN la regla route: SFTP falla**Dónde / comando:**

srv1: sudo ufw status numbered (sin route al puerto 22)

cli: sftp -P 2222 sftp_2225727@192.168.56.10

Debe verse en la captura:

- Sin regla FWD al 22
- Timeout / Connection timed out

```
vagrant@srv1-2225727:~$ sudo ufw status numbered
Status: active

      To Action From
      --
[ 1] 22/tcp ALLOW IN Anywhere
[ 2] 192.168.50.2 21/tcp ALLOW FWD Anywhere
[ 3] 192.168.50.2 50000:50010/tcp ALLOW FWD Anywhere
[ 4] 192.168.50.2 22/tcp ALLOW FWD Anywhere
[ 5] 22/tcp (v6) ALLOW IN Anywhere (v6)
```

```
vagrant@cli-2225727:~$ sftp -P 2222 sftp_2225727@192.168.56.10
^Cvagrant@cli-2225727:~$ |
```

CAPTURA 16.4 — Puerto 22 del Servidor 2 inaccesible directamente**Dónde / comando:**

cli: nc -zv -w3 192.168.50.2 22

Debe verse en la captura:

- Fallo / timeout

```
vagrant@cli-2225727:~$ nc -zv -w3 192.168.50.2 22
^C
vagrant@cli-2225727:~$ |
```

Punto 17 (0.2) — Conexión SFTP desde el cliente y huella de la clave de host

CAPTURA 17.1 — Primera conexión: huella de la clave de host

Mensaje The authenticity of host... con la huella.

Dónde / comando:

```
cli: sftp -P 2222 sftp_2225727@192.168.56.10
```

Debe verse en la captura:

- Tipo de clave (ED25519/ECDSA) y huella SHA256:...

```
vagrant@cli-2225727:~$ sftp -P 2222 sftp_2225727@192.168.56.10
The authenticity of host '[192.168.56.10]:2222 ([192.168.56.10]:2222)' can't be established.
ED25519 key fingerprint is SHA256:E08/uAcZPjcLEF7cg+AhKRjeZB+jsDMTH3Er7n5n0e0.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.56.10]:2222' (ED25519) to the list of known hosts.
sftp_2225727@192.168.56.10's password:
Connected to 192.168.56.10.
```

CAPTURA 17.2 — Huella real del servidor (comparación)

Use el mismo tipo de clave que mostró el cliente.

Dónde / comando:

```
srv2: ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
(o ssh_host_ecdsa_key.pub según el tipo)
```

Debe verse en la captura:

- Huella idéntica a la de la captura 17.1

```
vagrant@srv2-2225727:~$ ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:E08/uAcZPjcLEF7cg+AhKRjeZB+jsDMTH3Er7n5n0e0 root@ubuntu-jammy (ED25519)
```

CAPTURA 17.3 — Sesión: ls, put y get**Dónde / comando:**

```
sftp> ls
sftp> cd upload
sftp> put /etc/hostname 2225727.txt
sftp> ls
sftp> get 2225727.txt copia.txt
```

Debe verse en la captura:

- Salida de ls
- Mensajes Uploading ... y Fetching ...

```
sftp> pwd
Remote working directory: /
sftp> ls
upload
sftp> cd upload
sftp> put /tmp/2225727_sftp.txt
Uploading /tmp/2225727_sftp.txt to /upload/2225727_sftp.txt
2225727_sftp.txt
sftp> ls
2225727_sftp.txt
sftp> get 2225727_sftp.txt /tmp/2225727_sftp_descargado.txt
Fetching /upload/2225727_sftp.txt to /tmp/2225727_sftp_descargado.txt
2225727_sftp.txt
```

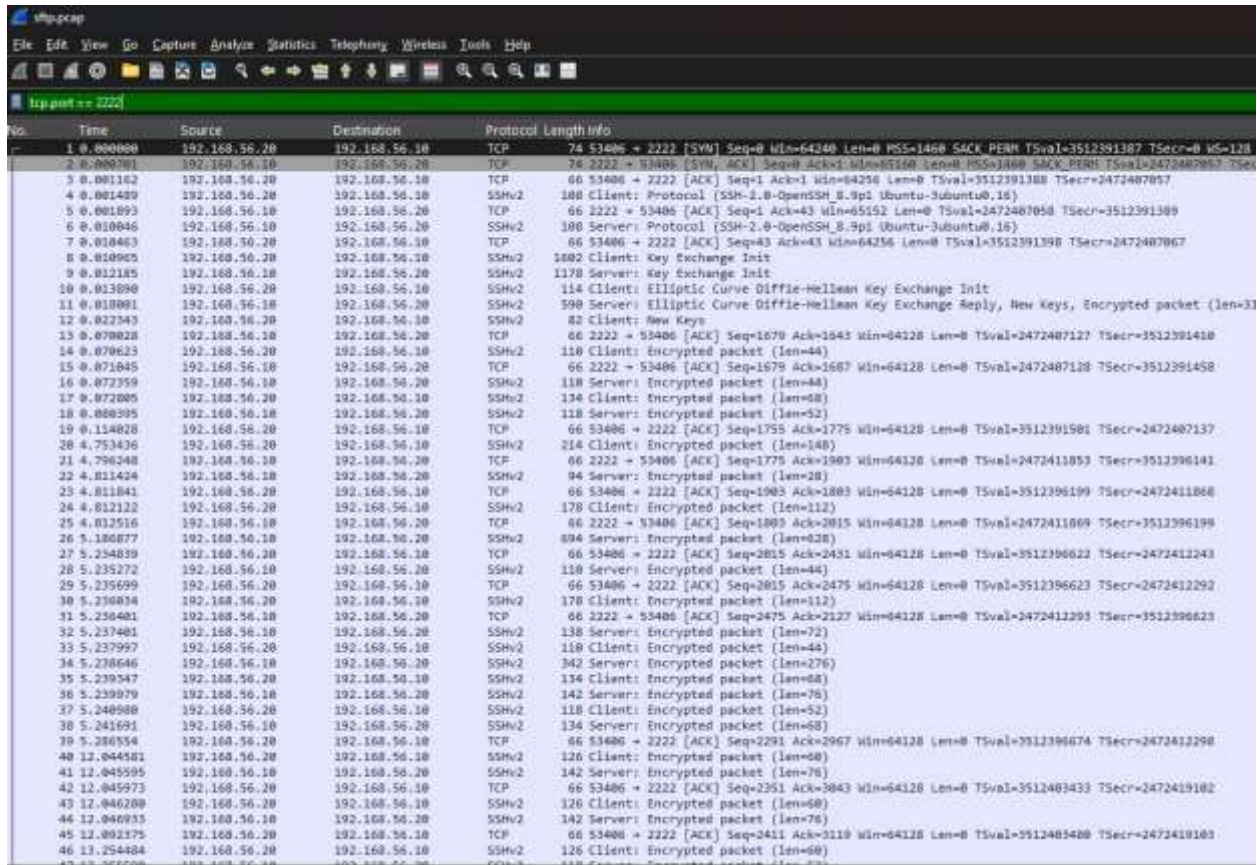
Punto 18 (0.3) — Wireshark de la sesión SFTP y comparación

CAPTURA 18.1 — Intercambio de versiones SSH-2.0

Filtro tcp.port == 2222.

Debe verse en la captura:

- Banner SSH-2.0-OpenSSH_... (cliente y servidor)



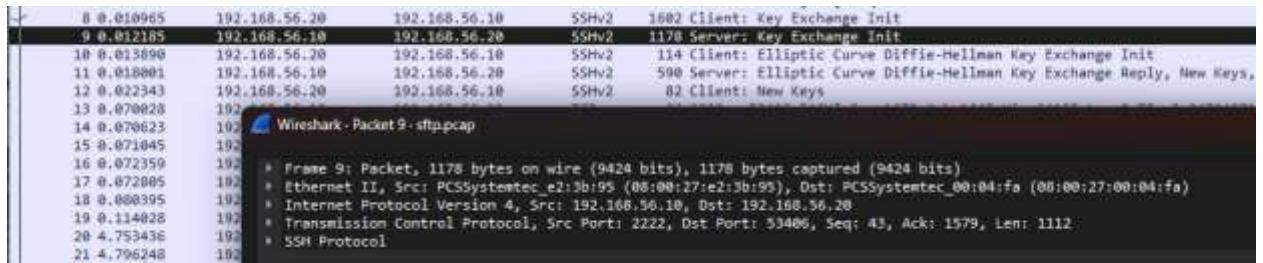
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.56.20	192.168.56.10	TCP	74	53406 → 2222 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3512391387 TSecr=0 WS=128
2	0.000791	192.168.56.10	192.168.56.20	TCP	74	2222 → 53406 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2472407057 TSecr=0
3	0.001162	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=1 Ack=1 Win=64240 Len=0 TSval=3512391388 TSecr=2472407057
4	0.001489	192.168.56.20	192.168.56.10	SSHv2	108	Client: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.16)
5	0.001693	192.168.56.10	192.168.56.20	TCP	66	2222 → 53406 [ACK] Seq=1 Ack=43 Win=65512 Len=0 TSval=2472407058 TSecr=3512391389
6	0.010046	192.168.56.10	192.168.56.20	SSHv2	108	Server: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.16)
7	0.010463	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=43 Ack=43 Win=64240 Len=0 TSval=3512391390 TSecr=2472407067
8	0.010903	192.168.56.20	192.168.56.10	SSHv2	108	Client: Key Exchange Init
9	0.012185	192.168.56.10	192.168.56.20	SSHv2	1178	Server: Key Exchange Init
10	0.013090	192.168.56.20	192.168.56.10	SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
11	0.013801	192.168.56.10	192.168.56.20	SSHv2	398	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys, Encrypted packet (len=31)
12	0.022343	192.168.56.20	192.168.56.10	SSHv2	82	Client: New Keys
13	0.070028	192.168.56.20	192.168.56.10	TCP	66	2222 → 53406 [ACK] Seq=1679 Ack=1643 Win=64128 Len=0 TSval=2472407127 TSecr=3512391410
14	0.070623	192.168.56.20	192.168.56.10	SSHv2	118	Client: Encrypted packet (len=44)
15	0.071045	192.168.56.10	192.168.56.20	TCP	66	2222 → 53406 [ACK] Seq=1679 Ack=1687 Win=64128 Len=0 TSval=2472407128 TSecr=3512391458
16	0.072359	192.168.56.10	192.168.56.20	SSHv2	118	Server: Encrypted packet (len=44)
17	0.072805	192.168.56.20	192.168.56.10	SSHv2	134	Client: Encrypted packet (len=68)
18	0.088305	192.168.56.10	192.168.56.20	SSHv2	118	Server: Encrypted packet (len=52)
19	0.114028	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=1755 Ack=1775 Win=64128 Len=0 TSval=3512391501 TSecr=2472407137
20	4.753436	192.168.56.20	192.168.56.10	SSHv2	214	Client: Encrypted packet (len=140)
21	4.796248	192.168.56.10	192.168.56.20	TCP	66	2222 → 53406 [ACK] Seq=1775 Ack=1903 Win=64128 Len=0 TSval=2472411853 TSecr=3512396141
22	4.811424	192.168.56.10	192.168.56.20	SSHv2	94	Server: Encrypted packet (len=28)
23	4.811841	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=1903 Ack=1803 Win=64128 Len=0 TSval=3512396199 TSecr=2472411868
24	4.812122	192.168.56.20	192.168.56.10	SSHv2	178	Client: Encrypted packet (len=112)
25	4.812516	192.168.56.10	192.168.56.20	TCP	66	2222 → 53406 [ACK] Seq=1803 Ack=2015 Win=64128 Len=0 TSval=2472411869 TSecr=3512396199
26	5.130877	192.168.56.10	192.168.56.20	SSHv2	894	Server: Encrypted packet (len=828)
27	5.234839	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=2015 Ack=2431 Win=64128 Len=0 TSval=3512396622 TSecr=2472412243
28	5.235272	192.168.56.10	192.168.56.20	SSHv2	118	Server: Encrypted packet (len=44)
29	5.235699	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=2015 Ack=2475 Win=64128 Len=0 TSval=3512396623 TSecr=2472412292
30	5.236034	192.168.56.20	192.168.56.10	SSHv2	178	Client: Encrypted packet (len=112)
31	5.236401	192.168.56.10	192.168.56.20	TCP	66	2222 → 53406 [ACK] Seq=2475 Ack=2127 Win=64128 Len=0 TSval=2472412293 TSecr=3512396623
32	5.237401	192.168.56.10	192.168.56.20	SSHv2	138	Server: Encrypted packet (len=72)
33	5.237997	192.168.56.20	192.168.56.10	SSHv2	118	Client: Encrypted packet (len=44)
34	5.270646	192.168.56.10	192.168.56.20	SSHv2	342	Server: Encrypted packet (len=276)
35	5.239347	192.168.56.20	192.168.56.10	SSHv2	134	Client: Encrypted packet (len=68)
36	5.239979	192.168.56.10	192.168.56.20	SSHv2	142	Server: Encrypted packet (len=76)
37	5.240980	192.168.56.20	192.168.56.10	SSHv2	118	Client: Encrypted packet (len=52)
38	5.241691	192.168.56.10	192.168.56.20	SSHv2	134	Server: Encrypted packet (len=68)
39	5.286354	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=2291 Ack=2967 Win=64128 Len=0 TSval=3512396674 TSecr=2472412298
40	12.044581	192.168.56.20	192.168.56.10	SSHv2	126	Client: Encrypted packet (len=60)
41	12.045595	192.168.56.10	192.168.56.20	SSHv2	142	Server: Encrypted packet (len=76)
42	12.045973	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=2351 Ack=3043 Win=64128 Len=0 TSval=3512403433 TSecr=2472419182
43	12.046209	192.168.56.20	192.168.56.10	SSHv2	126	Client: Encrypted packet (len=60)
44	12.046935	192.168.56.10	192.168.56.20	SSHv2	142	Server: Encrypted packet (len=76)
45	12.092375	192.168.56.20	192.168.56.10	TCP	66	53406 → 2222 [ACK] Seq=2411 Ack=3119 Win=64128 Len=0 TSval=3512403480 TSecr=2472419183
46	13.254484	192.168.56.20	192.168.56.10	SSHv2	126	Client: Encrypted packet (len=60)

CAPTURA 18.2 — Intercambio de claves

Mismo filtro.

Debe verse en la captura:

- Key Exchange Init / Diffie-Hellman / New Keys



No.	Time	Source	Destination	Protocol	Length	Info
8	0.010965	192.168.56.20	192.168.56.10	SSHv2	1602	Client: Key Exchange Init
9	0.012185	192.168.56.10	192.168.56.20	SSHv2	1178	Server: Key Exchange Init
10	0.013890	192.168.56.20	192.168.56.10	SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
11	0.018001	192.168.56.10	192.168.56.20	SSHv2	590	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
12	0.022343	192.168.56.20	192.168.56.10	SSHv2	82	Client: New Keys

Wireshark - Packet 9 - sftp.pcap

Frame 9: Packet, 1178 bytes on wire (9424 bits), 1178 bytes captured (9424 bits)

Ethernet II, Src: PCSSystemtec_e2:3b:93 (08:00:27:e2:3b:93), Dst: PCSSystemtec_00:04:fa (08:00:27:00:04:fa)

Internet Protocol Version 4, Src: 192.168.56.10, Dst: 192.168.56.20

Transmission Control Protocol, Src Port: 2222, Dst Port: 53406, Seq: 43, Ack: 1579, Len: 1112

SSH Protocol

CAPTURA 18.3 — Todo cifrado en un único canal

Paquetes Encrypted packet, más Statistics → Conversations → pestaña TCP.

Debe verse en la captura:

- Paquetes Encrypted packet (len=...)
- Una sola conversación TCP hacia el puerto 2222

The screenshot displays the Wireshark interface with a capture of a TCP stream (tcp.stream eq 0). The packet list shows several packets, with the selected packet (No. 16) being an 'SSH-2.0-OpenSSH 8.9p1 Ubuntu-3ubuntu0.16' packet. The packet details pane shows the 'SSH' protocol and its various components, including 'SSH-2.0-OpenSSH 8.9p1 Ubuntu-3ubuntu0.16' and 'SSH-2.0-OpenSSH 8.9p1 Ubuntu-3ubuntu0.16'. The raw packet data pane shows the hexadecimal and ASCII representation of the packet data.

Comparación con las capturas de la Primera Parte (complete con lo que observó):

Aspecto	FTP plano	FTPS	SFTP
Nº de conexiones TCP	2: una para control y otra para datos durante la transferencia.	2: una para control y otra para datos en el rango pasivo.	1: autenticación y transferencia de archivos utilizan el mismo canal SSH.
Puertos usados	TCP 21 para control y puertos pasivos para datos.	TCP 21 para control y TCP 50000–50010 para datos pasivos.	TCP 2222 en srv1, reenviado por DNAT a TCP 22 de srv2.
Usuario/clave visibles	Sí, se observaron USER y PASS en Wireshark.	No , la autenticación ocurre dentro del canal TLS cifrado.	No, la autenticación está protegida por SSH.
Contenido de archivos visible	Sí, se pudo observar el contenido en texto plano mediante Wireshark.	No, los datos aparecen cifrados como Application Data.	No, los datos aparecen cifrados dentro del canal SSH.
Cuándo inicia el cifrado	No existe cifrado.	Después del comando AUTH TLS, seguido del handshake TLS.	Durante el establecimiento de la conexión SSH, antes de transmitir autenticación y datos de usuario.

RESPUESTA — Conclusión de la comparación

Resuma qué es visible y cuántas conexiones/puertos hay en cada caso.

En FTP plano, tanto las credenciales como el contenido de los archivos pueden observarse directamente en la captura. FTPS cifra el canal de control después de AUTH TLS y también cifra el canal de datos, pero requiere conexiones y puertos adicionales para el modo pasivo. SFTP utiliza un único canal SSH cifrado, por lo que la autenticación y los datos no quedan expuestos y la comunicación resulta más sencilla de controlar mediante el firewall.

Punto 19 (0.3) — Tabla comparativa FTPS vs SFTP y conclusión

Criterio	FTPS	SFTP
Protocolo base	FTP protegido mediante TLS.	SSH (Secure Shell).
Nº de conexiones y puertos	Canal de control en TCP 21 y canal de datos mediante puertos pasivos 50000–50010.	Un único canal TCP; puerto externo 2222 en srv1, reenviado a TCP 22 de srv2.

Criterio	FTPS	SFTP
Autenticación del servidor (X.509 vs clave de host SSH)	Certificado digital X.509 presentado durante TLS.	Clave de host SSH.
Momento en que inicia el cifrado	Después de AUTH TLS, mediante el handshake TLS.	Durante el establecimiento de la conexión SSH, antes de la autenticación y transferencia de datos. Durante el establecimiento de la conexión SSH, antes de la autenticación y transferencia de datos.
Facilidad para atravesar firewalls/NAT	Más compleja porque requiere el puerto 21 y un rango adicional de puertos pasivos, además de DNAT y reglas de forwarding.	Más sencilla porque todo el tráfico utiliza un único canal y un único puerto externo, 2222.
Facilidad de configuración	Más compleja debido a certificados TLS, pasv_address, rango pasivo y reglas adicionales de firewall/NAT.	Más sencilla, ya que OpenSSH integra autenticación, cifrado y transferencia en un solo canal.

RESPUESTA — Conclusión: ¿cuál es más adecuado con restricciones estrictas de firewall y por qué?

Base su respuesta en sus propias evidencias (cantidad de reglas DNAT/route necesarias, puertos abiertos, pasv_address, etc.).

SFTP es más adecuado para un entorno con restricciones estrictas de firewall, debido a que utiliza un único canal SSH cifrado. En nuestra implementación fue suficiente publicar el puerto 2222 en srv1 y reenviarlo mediante DNAT al puerto 22 de srv2, junto con una única regla ufw route para el puerto 22. En cambio, FTPS requirió el puerto 21, un rango de puertos pasivos 50000–50010, reglas DNAT y reglas de forwarding adicionales, además de configurar pasv_address. Por lo tanto, SFTP presenta una configuración más sencilla para atravesar firewalls y NAT y mantiene tanto la autenticación como los datos protegidos en el mismo canal.